



Fundamentos da Programação Orientada a Objetos

Componentes de entrada e saída

Nesta última unidade da disciplina, serão apresentadas algumas alternativas para **troca de dados** entre nossos programas e outros elementos. Isso, na prática, significa que aprenderemos sobre:

- **Manipulação de arquivos para escrita (ou saída) e leitura (ou entrada) de dados usando Java:**

As informações podem ser armazenadas (escritas) e recuperadas (lidas) de outros elementos (como arquivos) graças a um sistema de comunicação (troca de dados), conhecido como fluxo (*stream*). Esta troca pode ser feita com **dados primitivos**, como *bytes*, ou com **objetos**.

- **Interface gráfica, para interação por eventos, usando Java:**

Eventos são **ocorrências** que disparam alguma funcionalidade, como quando acionamos um botão em uma interface gráfica e aparece uma janela com alguma mensagem.

Vamos lá?

| Leitura e gravação de dados em arquivo

Nos nossos programas, manipulamos dados por meio de variáveis, que podem ser tipos primitivos, vetores, objetos etc. Contudo, esse armazenamento de dados é temporário. Afinal, tais **dados se perdem** quando a **variável deixa de existir**, ao ser finalizado o programa ou ao sair de uma função ou método que utiliza temporariamente aquela variável.

Para **manter nossos dados a longo prazo**, precisamos persisti-los, o que significa que eles serão gravados em meio não volátil (como arquivos), para então, serem recuperados em outro momento.



IMPORTANTE

Persistência de dados garante de que um dado foi salvo (escrito) e que poderá ser recuperado (lido) em um outro momento.

Na computação, **persistência** significa que o dado (estado do programa) sobrevive ao programa no qual ele foi criado; dados persistidos são chamados de dados acessíveis a longo prazo.

Representação binária: no computador, todo dado, seja ele um texto, um arquivo de som, um arquivo de imagem, ou qualquer outro tipo de arquivo, é representado como um **conjunto de bits**, que são representados por números zeros (0) e uns (1). É o que chamamos de representação binária, por ter apenas **dois** símbolos.

Por essa razão, quando nossos programas leem um dado, eles recebem sua representação binária.

Exemplo: **letra maiúscula A**: código decimal = 65 e código binário = 01000001 (conjunto de **bits**).

É o **código binário** da letra **A** que fica nas variáveis dos nossos programas, que é transportado pela rede, que é armazenado em arquivo, que é passado pelo teclado, e assim por diante. **No computador, tudo é binário.**

São os nossos programas que manipulam esses **códigos binários em bits** para apresentar as informações **adequadas e esperadas**, como texto, som, imagem etc.

O **bit** é a menor informação que o computador pode armazenar.

- Comumente, usado para medir a velocidade da internet, como **100 Mbps** (megabits $\cong 100 \times 10^6$ bits por segundo); embora a internet forneça dados em bytes, eles são transportados pela rede de bit em bit, um por vez, em série.

O **byte** é uma unidade de memória que contém **oito bits**:

- São usados para especificar a quantidade de memória ou a capacidade de armazenamento de um certo dispositivo, como **8 GB** (giga bytes $\cong 8 \times 10^9$ bytes) de RAM, **1TB** (tera bytes $\cong 1 \times 10^{12}$ bytes) de disco rígido etc.

Agora que identificamos bits e bytes, veja como esses dados binários são **armazenados** e **recuperados** por meio de um sistema de comunicação conhecido como fluxo (stream). Detalhes na Figura 1 a seguir.

FLUXO (*STREAM*) - representa uma **origem** (*DATA SOURCE*) ou um **destino de dados** (*DATA DESTINATION*), que pode ser:

- Arquivos em disco.
- Dispositivos físicos (monitor, teclado etc.).
- Outros programas.
- Um socket de rede*, etc.

O *stream* também suporta **diferentes tipos de dados**:

- Bytes.
- Vetores.
- Tipos de dados heterogêneos.
- Até **objetos**.

* Ponta final de um fluxo de comunicação entre processos por meio de uma rede de computadores

#ParaTodosVerem 

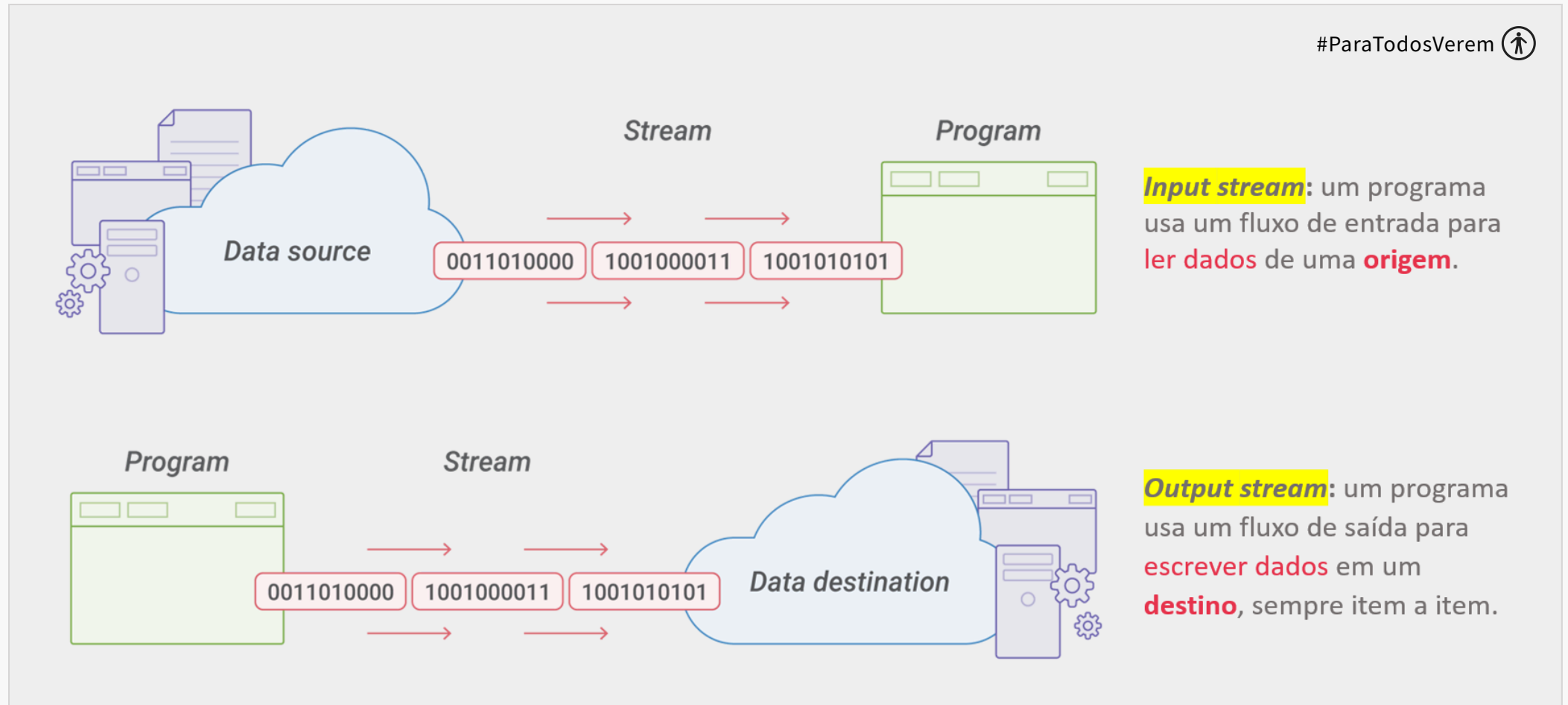


Figura 1: Fluxo (stream). Fonte: Autores (2021).



EXEMPLO

Exemplo de aplicação 1: fluxo de bytes com `FileOutputStream` e `FileInputStream`

Existem várias classes de fluxos de byte, e todas derivadas de `OutputStream` / `InputStream`. Na prática, você já estava trabalhando com entrada e saída de bytes sem perceber. Veja só:

- Objeto `System.out` é um `OutputStream` para escrever na **tela**. O `System.out.println()` vem daqui.
- Objeto `System.in` é um `InputStream` para ler dados do **teclado**. O `new Scanner(System.in)` usa isso para ler os dados do teclado.

Neste exercício, apresentamos como você pode usar as classes `FileOutputStream` e `FileInputStream` para **mandar** e **receber** dados de um arquivo, *byte a byte*, ou de oito em oito bits. Observe que essas classes **ocultam** como a manipulação binária é feita – apenas precisamos saber como utilizá-las. Geralmente, usamos essas duas classes para a **entrada/saída** de arquivos em *bytes* (8 bits).

Dados de um fluxo podem também vir de arquivos:

Formato geral para **leitura** de dados - `FileInputStream` (ler)

```
FileInputStream arqIn = new FileInputStream (“dadosIn.dat”);  
...  
arqIn.close(); //Após leitura, um arquivo deve ser fechado com o método close
```

Formato geral para **escrita** de dados - `FileOutputStream` (escrever)

```
FileOutputStream arqOut = new FileOutputStream(“dadosOut.dat”);  
...  
arqOut.close(); //Após escrita, um arquivo deve ser fechado com o método close
```

Em um novo projeto Java, crie uma classe chamada **Byte**. Insira o código a seguir no arquivo **Byte.java**, e execute-o:

```
</>
1  import java.io.FileInputStream;
2  import java.io.FileNotFoundException;
3  import java.io.FileOutputStream;
4  import java.io.IOException;
5
6  public class Byte {
7      public static void EscreverDados() // escreve bytes para um arquivo
8      {
9          // o try-with-resources fecha o 'arq' automaticamente no final
10         try (FileOutputStream arq = new FileOutputStream("arq1.txt")) {
11             //Do caractere A (código 65) ao Z (código 90)
12             for(int cont = 65; cont < 91; cont++) {
13                 arq.write(cont); // Escreve Byte (8bits) em arquivo
14                 System.out.print((char)cont); // imprime o código como char
15             }
16         } catch (FileNotFoundException e) {
17             e.printStackTrace();
18         } catch (IOException e) {
19             e.printStackTrace();
20         }
21     }
22 }
```

Perceba as linhas 12-15: escreveremos todos os caracteres a partir da letra A maiúscula (valor 65) até a letra Z maiúscula (valor 90) em um arquivo chamado “arq1.txt” (linha 10). Depois, leremos os dados desse mesmo arquivo (linha 28) e mostraremos o seu teor na tela. Finalmente, copiaremos o teor do “arq1.txt” para o “arq2.txt” (linhas 46-54). Você também verá isto na tela:

Escrita-----
ABCDEFGHIJKLMNOPQRSTUVWXYZ


```
Leitura-----  
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

```
Copia-----  
ABCDEFGHIJKLMNOPQRSTUVWXYZ  
Process finished with exit code 0
```

Quer entender melhor os códigos mais básicos? Veja a tabela a seguir: consulte o caractere que deseja ler (exemplo: letra maiúscula “A” na coluna “Char”). Depois, veja o seu valor correspondente na coluna “Decimal”. Veja que o valor de “A” é 65.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Fonte: Domínio público: <https://pt.m.wikipedia.org/wiki/Ficheiro:ASCII-Table-wide.svg>



EXEMPLO

Exemplo de aplicação 2: fluxo de caracteres

Neste exercício, apresentamos como utilizar as **classes** `FileWriter` e `FileReader` para **mandar** e **receber** dados de um arquivo, caractere a caractere, ou de 16 em 16 bits. No caso, essas classes possuem os seguintes usos:

- `FileReader`, para fluxo de caracteres de **entrada**.
- `FileWriter`, para fluxo de caracteres de **saída**.

Os fluxos de caracteres são um tipo **especializado** de fluxo de bytes para tratar caracteres **unicode**, um **padrão** para representar e manipular texto dos sistemas de escrita existente. Com o padrão unicode, é possível codificar cerca de 138 mil caracteres diferentes ([Wikipedia](#)). Ele foi construído a partir da tabela ASCII de 128 valores que vimos anteriormente, mas foi adaptado para diferentes sistemas de escrita, símbolos e emojis.

Veja, no código, o método `read ( c = in.read() )` retorna um **int** tanto para o fluxo de bytes quanto para o fluxo de caracteres.

Contudo:

- **Fluxo de bytes**, o **inteiro** retornado = 8 bits de tamanho.
- **Fluxo de caracteres**, o **inteiro** retornado = 16 bits de tamanho.

Lendo e escrevendo linhas

Para manipular arquivos de textos, com informações separadas por **finalizador de linha** (`\r`, `\n`, `\r\n`), usamos as classes `BufferedReader` para ter uma melhor eficiência nas operações de leitura/escrita de dados, que geralmente são **lentas**, pois acessam o disco.

Para isso, um **buffer** (área temporária de memória) é usado para acumular leituras ou escritas.

Além disso, utilizamos também a classe *PrintWriter* para ter mais conforto na escrita do texto. Você já a conhece, pois é a mesma utilizada pelo *System.out*. Ela fornece vários métodos convenientes para a escrita de texto, como o *println* e o *printf*, usados nas unidades passadas. Vários fluxos de dados permitem especificar um fluxo de destino em seu construtor. Isso fornece uma forma flexível de combiná-los.

Analise o código do exemplo a seguir, que escreve, lê caracteres em arquivo e copia, **linha a linha**, dados de um **arquivo texto** para outro. Preste especial atenção sobre como combinamos os fluxos entre si.

Execute o código a seguir em um novo projeto em Java. O nome da classe será **CopiarCaracter**.

```
</>
```

```

1  import java.io.BufferedReader;
2  import java.io.FileNotFoundException;
3  import java.io.FileReader;
4  import java.io.FileWriter;
5  import java.io.IOException;
6  import java.io.PrintWriter;
7
8  public class CopiarCaracter {
9      public static void EscreverCaracteres(){
10         FileWriter out = null;
11         int contLetra = 0;
12         String texto = "Texto para gravar no arq.\nOutro texto para gravar no arq.";
13         try {
14             out = new FileWriter("arqChar1.txt");
15             while (contLetra < texto.length()){
16                 out.write(texto.charAt(contLetra)); // escreve caractere a caractere
17                 contLetra++;
18             }
19             out.close(); // fecha arquivo de saída

```

Você também verá isto na tela:

Escrita-----

Leitura-----

Texto para gravar no arq.

Outro texto para gravar no arq.

Copia-----

Texto para gravar no arq.

Outro texto para gravar no arq.

O código apresentado é um exemplo de manipulação de arquivos em Java. Ele consiste em três métodos distintos que realizam as operações de escrita, leitura e cópia de arquivos, respectivamente. Vamos abordar cada um deles:

1. **EscreverCaracteres()**: este método escreve texto em um arquivo chamado "arqChar1.txt". O texto é definido na string "texto", e o método escreve caractere por caractere no arquivo usando um loop **while**. Se o arquivo não existir, ele será criado. Caso o arquivo já exista, o conteúdo anterior será substituído pelo novo texto. Exceções são tratadas por meio dos blocos **catch** para **FileNotFoundException** e **IOException**.
2. **LerCaracteres()**: este método lê o arquivo "arqChar1.txt" criado pelo método anterior. Ele lê e imprime o conteúdo do arquivo caractere por caractere usando um loop **while**. Semelhantemente ao método anterior, exceções são tratadas nos blocos **catch** para **FileNotFoundException** e **IOException**.
3. **CopiarLinha()**: este método faz uma cópia do conteúdo do arquivo "arqChar1.txt" e grava em um novo arquivo chamado "arqCharLinha2.txt". Ele usa a classe **BufferedReader** para ler o conteúdo do arquivo de origem e a classe **PrintWriter** para escrever no arquivo de destino. Aqui, a leitura e a escrita são feitas linha por linha, em vez de caractere por caractere.

Finalmente, no método **main()**, os três métodos são chamados em sequência. Primeiramente, o texto é escrito no arquivo "arqChar1.txt" por meio do método **EscreverCaracteres()**. Depois, o conteúdo do arquivo é lido e impresso na tela pelo método **LerCaracteres()**. E por último, o conteúdo do arquivo é copiado para o arquivo "arqCharLinha2.txt" pelo método **CopiarLinha()**. Durante a cópia, cada linha lida do arquivo de origem também é impressa na tela.

| Persistência de objeto com controle de versão

Assim como conseguimos gravar e recuperar **linhas inteiras de arquivos-texto**, também, conseguimos gravar e recuperar **objetos**.

Para isso, precisamos primeiramente transformar a estrutura do objeto em uma sequência binária. Fazemos isso serializando o estado do objeto (seus atributos) em uma stream (cadeia de bytes), que pode então ser **gravado em arquivo binário** ou **transportado por meio de uma rede**.

E, para poder ser serializada, uma classe precisa **implementar a interface** `java.io.Serializable` ou ser **herdeira de uma classe que a implemente**.



IMPORTANTE

Classes que não implementam a interface `java.io.Serializable` não podem **serializar/desserializar** seu estado. Ou seja, **seus objetos não podem ser persistidos** ou transferidos pela rede.

Controle de versão do objeto serializado

Durante a serialização, o Java Runtime (ambiente de execução Java) associa um identificador, que é um número de versão, a cada **classe serializável**.

Esse número, denominado serialVersionUID, é usado durante a **desserialização**, para verificar se o remetente e o receptor de um **objeto serializado** carregaram **classes compatíveis desse objeto**.

Para melhor observar esses conceitos, vamos praticar a persistência de objetos, no próximo exercício.



EXEMPLO

Exemplo de aplicação 3: gravando e recuperando objetos com serialização.

Crie um projeto em Java. O seu projeto deverá possuir os arquivos com os códigos listados a seguir. Perceba o uso do `implements Serializable` na linha 2: é ela que ajuda a determinar a serialização da classe.

Arquivo Pessoa.java:

```
</>
1  import java.io.Serializable;
2  public class Pessoa implements Serializable { // Classe comum, mas serializável!
3
4      // Adiciona um ID de versão serial padrão a classe.
5      private static final long serialVersionUID = 1L;
6
7      private String nome;
8      private String sobrenome;
9      private int idade;
10
11     public String getNome() {
12         return nome;
13     }
14     public void setNome(String firstName) {
15         this.nome = firstName;
16     }
17     public String getSobrenome() {
18         return sobrenome;
19     }
```

Arquivo Objetos.java:

</>

```
1 import java.io.EOFException;
2 import java.io.FileInputStream;
3 import java.io.FileNotFoundException;
4 import java.io.FileOutputStream;
5 import java.io.IOException;
6 import java.io.ObjectInputStream;
7 import java.io.ObjectOutputStream;
8
9 public class Objetos {
10
11     public static void escrevePessoas(String filename) {
12         ObjectOutputStream outputStream = null;
13         try {
14             outputStream = new ObjectOutputStream (new FileOutputStream(filename));
15
16             Pessoa person = new Pessoa();
17             person.setNome("James");
18             person.setSobrenome("Ryan");
19             person.setIdade(19);
```

Agora, execute o código a partir de **Objetos.java**. Você verá na saída do console o nome de dois personagens e, finalmente, o texto “Fim de arquivo alcançado”, não é? Usamos a classe `ObjectOutputStream` para **escrever objetos** e a classe `ObjectInputStream` para **ler objetos** neste exemplo. Mais especificamente, nos métodos **escrevePessoas()** e **lePessoas()**. Observe que a classe dos objetos que são salvos e recuperados **implementa** a interface `java.io.Serializable`. No caso, a classe do objeto que estamos serializando (gravando) é **Pessoa**.

Agora, altere a linha 3 da classe `Pessoa`, retirando a implementação da interface `Serializable`. Teste o código novamente. Você verá que, durante a compilação do código, é gerado o erro: **`java.io.NotSerializableException: Objetos.Pessoa`**. Este é o ponto da serialização: não é possível gravar um objeto que não pode ser serializado.

Bom, analise novamente o código e o seu resultado. Quantas e quais instâncias do objeto foram gravadas?

De fato, foram duas instâncias, a saber: ("James Ryan", age=19) e ("Obi-wan Kenobi", age=30). Você pode ver isso nas linhas 16 a 26 no arquivo **Obejtos.java**.

Agora, experimente remover o try/catch da leitura e escrita de dados. No caso, o seu código dos métodos **lePessoas()** e **escrevePessoas()** ficaria conforme o exemplo a seguir. Teste o código novamente.

```
</>

1  import java.io.EOFException;
2  import java.io.FileInputStream;
3  import java.io.FileNotFoundException;
4  import java.io.FileOutputStream;
5  import java.io.IOException;
6  import java.io.ObjectInputStream;
7  import java.io.ObjectOutputStream;
8
9  public class Objetos {
10
11      public static void escrevePessoas(String filename) {
12          ObjectOutputStream outputStream = null;
13          outputStream = new ObjectOutputStream (new FileOutputStream(filename));
14
15          Pessoa person = new Pessoa();
16          person.setNome("James");
17          person.setSobrenome("Ryan");
18          person.setIdade(19);
19          outputStream.writeObject(person); //só escreve pq Pessoa é serializável
```

Percebeu que houve um erro de compilação do tipo `UnhandledException`, não é? Isso significa que este código modificado (mais especificamente a manipulação de arquivo) exige tratamento de *try-catch*.

Para finalizar, o que acha de incluir mais duas pessoas no método `escrevePessoas()`? Um exemplo seria as linhas 28 a 38, a seguir:

```
</>
1  import java.io.EOFException;
2  import java.io.FileInputStream;
3  import java.io.FileNotFoundException;
4  import java.io.FileOutputStream;
5  import java.io.IOException;
6  import java.io.ObjectInputStream;
7  import java.io.ObjectOutputStream;
8
9  public class Objetos {
10
11      public static void escrevePessoas(String filename) {
12          ObjectOutputStream outputStream = null;
13          try {
14              outputStream = new ObjectOutputStream (new FileOutputStream(filename));
15
16              Pessoa person = new Pessoa();
17              person.setNome("James");
18              person.setSobrenome("Ryan");
19              person.setIdade(19);
```

Interfaces gráficas, também conhecidas como Graphical User Interfaces (GUI), são uma maneira intuitiva de interagir com programas de computador. Em vez de escrever comandos em uma linha de texto, o usuário pode interagir diretamente com elementos gráficos na tela, como botões, menus e caixas de texto.

As interfaces gráficas funcionam com base de **disparo de eventos**. Nesse modelo, o fluxo do programa é determinado por eventos, como cliques de mouse, pressionamento de teclas, movimento do mouse etc. Vamos tomar como exemplo um botão em uma interface gráfica. O botão é um objeto na tela que o usuário pode interagir, não é? Quando o usuário clica no botão, um **evento de clique** é disparado. Esse evento é uma espécie de sinal que informa ao programa que algo aconteceu.

Para que algo útil aconteça quando o usuário clica no botão, o programa deve ter um "ouvinte de eventos" (event listener) configurado para esse botão. Um ouvinte de eventos é um pedaço de código que "ouve" um tipo específico de evento e executa um bloco de código quando esse evento ocorre.

Por exemplo, se você tiver um botão "Enviar" em um aplicativo de e-mail, você configuraria um ouvinte de eventos para escutar o evento de clique desse botão. Quando o botão "Enviar" é clicado, o ouvinte de eventos reconhece que o evento ocorreu e executa o código que você configurou para ser executado quando o botão é clicado – neste caso, o código poderia ser para enviar o e-mail.

Na prática, existem processos implícitos nos nossos programas gráficos que se encarregam de capturar determinados eventos e executar os códigos de programas, que são as respostas para estes eventos.



IMPORTANTE

A **ocorrência de um evento** pode provocar uma reação que pode ser uma ação (ou um conjunto delas) a ser tomada – de acordo com o que nós programadores definimos.

Para criar a **nossa interface gráfica**, usaremos algumas funcionalidades do **Java Swing** que, como tudo no Java, é independente de plataforma. Isso significa que, com o Swing, nossos programas têm uma aparência muito parecida, independentemente do sistema operacional, ou plataforma, em que está sendo utilizado. Sua biblioteca está no pacote `javax.swing`, que provê classes para a API Java Swing API, como `JButton`, `TextField`, `TextArea`, `JRadioButton`, `JCheckbox`, `JMenu`, `JColorChooser` etc. Veja como eles são utilizados na Figura 2 a seguir.

Figura 2: Alguns componentes Java Swing.

JFrame representa a janela principal do programa com barra de título

TextField campo de texto, que pode receber texto de 1 linha

PasswordField campo de texto, para capturar texto protegido (senha)

CheckBox caixa de seleção (seleciona ou não).

RadioButton caixa de seleção (seleciona entre várias opções)

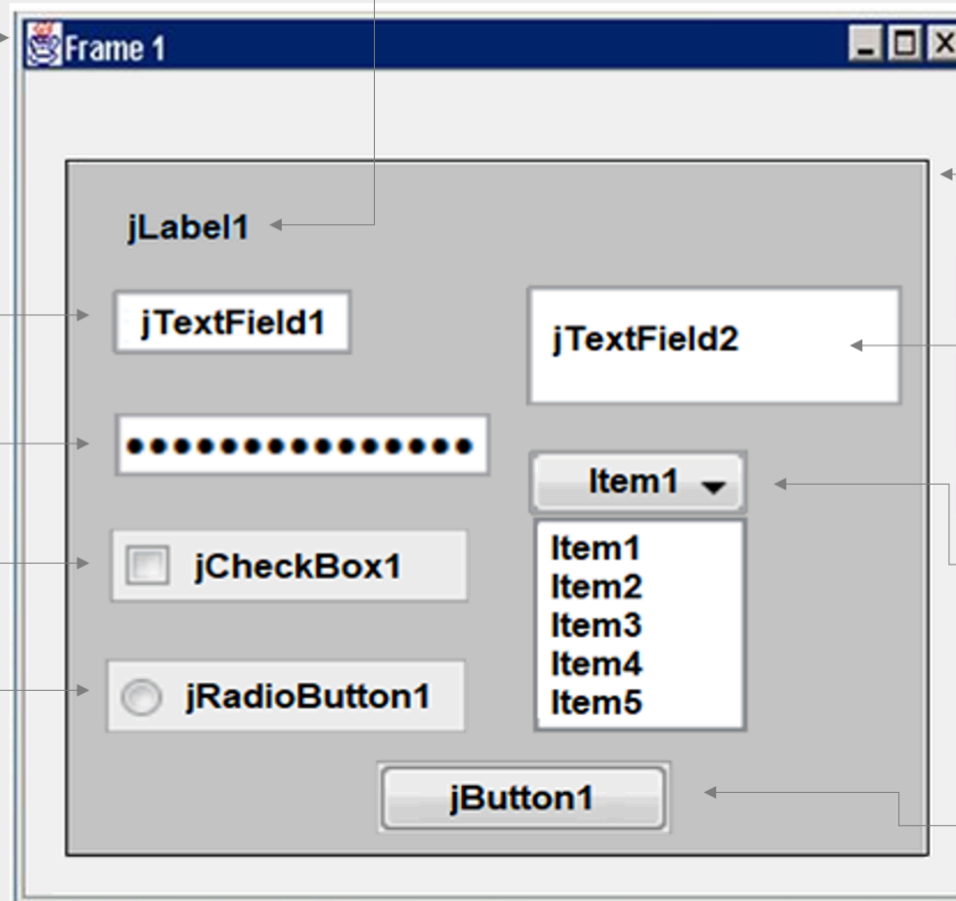
JLabel representa um rótulo de texto.

JPanel container onde colocamos nossos componentes

TextField campo de texto, que pode receber texto de 1 linha

ComboBox lança menu pop up com opções

Button botão que aciona ação



O **Swing** é parte do Java Foundation Classes (**JFC**) (<https://www.oracle.com/java/technologies/java-foundation-classes.html>), um conjunto abrangente de componentes e serviços de **GUI**, ou interface gráfica do usuário, que simplificam bastante o desenvolvimento de aplicações gráficas para desktop, principalmente com apoio de um IDE.

- O IDE **NetBeans** (<https://netbeans.org/>) fornece vários recursos para trabalhar com aplicações gráficas com Swing. Mais detalhes podem ser encontrados em: [Projetando uma GUI Swing no NetBeans IDE](#).
- O **IntelliJ** também possui capacidade para o design de interfaces em: [Designing GUI. Major Steps \(em inglês\)](#).

Exploraremos algumas funcionalidades do Swing que vão nos permitir criar uma aplicação simples, mas que já permitem interagir com o usuário via pequenas janelas, caixas de texto e botões.

Porém, antes, precisamos conhecer a classe *JOptionPane*. Vamos fazer isso na próxima prática, com o exercício apresentado a seguir.



EXEMPLO

Exemplo de aplicação 4: interface gráfica com Swing.

A classe `javax.swing.JOptionPane` é usada para fornecer **caixas de diálogo padrão** (pequenas janelas de pop-up), como as obtidas com os métodos estáticos:

1. `JOptionPane.showMessageDialog( componentePai , "mensagem" )` – cria uma caixa de diálogo para exibir uma mensagem.
2. `JOptionPane. showInputDialog( componentePai , "mensagem" ) – cria uma caixa de diálogo que também solicita um valor de entrada, digitada pelo usuário.`

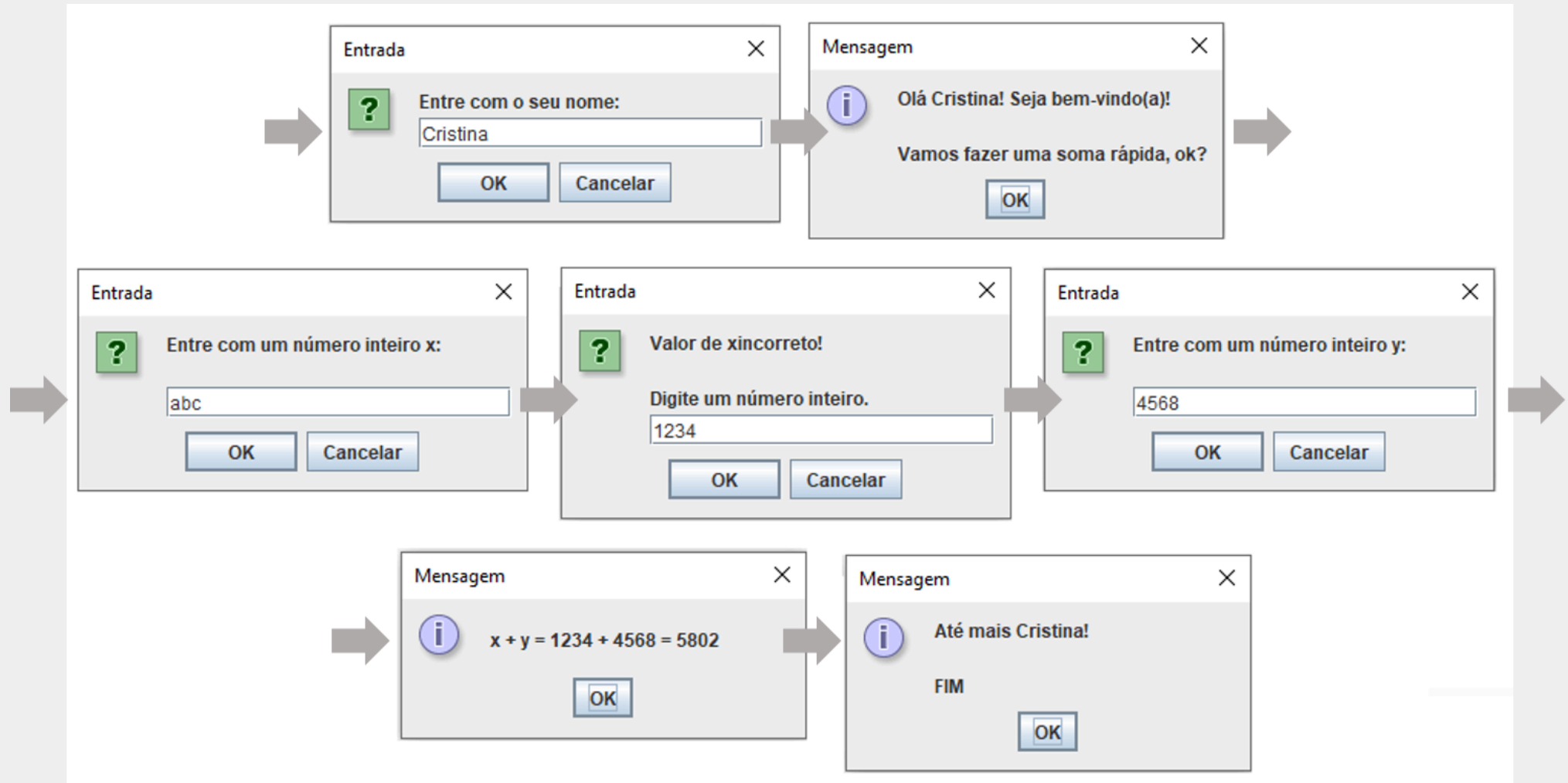
Em uma aplicação gráfica mais completa, é usual criar um objeto de `javax.swing.JFrame` para ser usado como **componentePai** para as caixas de diálogo da `JOptionPane`.

No exemplo da nossa aplicação simples, usamos `null` como `componentePai`. Isso significa que as **nossas caixas de diálogo** são “órfãs”, ou não estão vinculadas a nenhuma outra janela.

Verifique se você consegue obter a mesma sequência de caixas de diálogo do exercício.

```
</>
1  import javax.swing.JOptionPane; // Biblioteca da classe JOptionPane
2
3  public class MinhaInterface {
4      private boolean numeroInteiroValido(String s) {
5          boolean resultado;
6          try {
7              Integer.parseInt(s); // Tenta transformar uma string em inteiro
8              resultado = true;
9          } catch (NumberFormatException e) { // Gera erro se não consegue
10             resultado = false;
11         }
12         return resultado;
13     }
14     public int inteiroValido(String nomeVar) { // retorna um valor inteiro
15         int numInt;
16         String entrada;
17
18         entrada = JOptionPane.showInputDialog (null, "Entre com um número inteiro " + nomeVar + ":\n\n");
19     }
20 }
```

Sequência de apresentação das caixas de diálogo do programa



Fonte: Autores (2021).

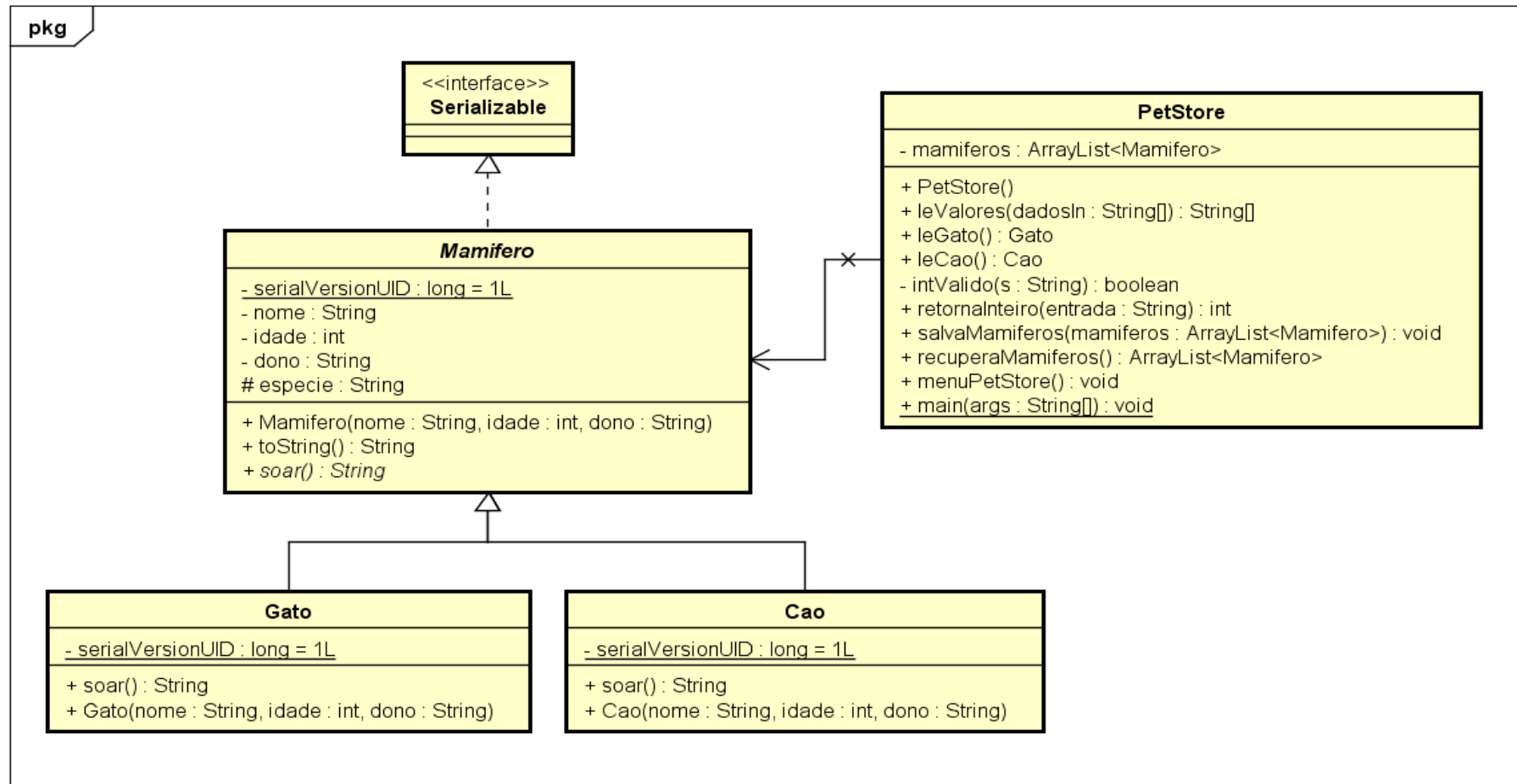


EXEMPLO

Exemplo de aplicação 5: persistindo objetos com interface gráfica.

Implemente as classes representadas no diagrama a seguir, conforme as definições expostas.

#ParaTodosVerem 



Fonte: Autores (2021).

Para executar esse programa, você deve:

1. Observar as relações de **herança** (Java `extends`) e **realização** (Java `implements`) do diagrama UML para completar os códigos das classes cao, gato e mamifero – procurar e trocar no código os seguintes comentários: `//complete o código aqui`.

2. Após completar os códigos e conseguir executar, acrescente mais as classes cavalo e leão, que fazem relinchos e rugidos, respectivamente. Altere os códigos para que você consiga incluir todos esses **quatro tipos** de mamíferos.

</>

```
1 public class Cao /*complete o código aqui */ {
2
3     private static final long serialVersionUID = 1L;
4
5     public String soar() {
6         return "Faz latidos";
7     }
8     //complete o código aqui
9     //complete o código aqui
10 }
11 }
```

</>

```
1 public class Gato /*complete o código aqui */ {
2     //complete o código aqui
3     // ...
4     //complete o código aqui
5 }
```

</>

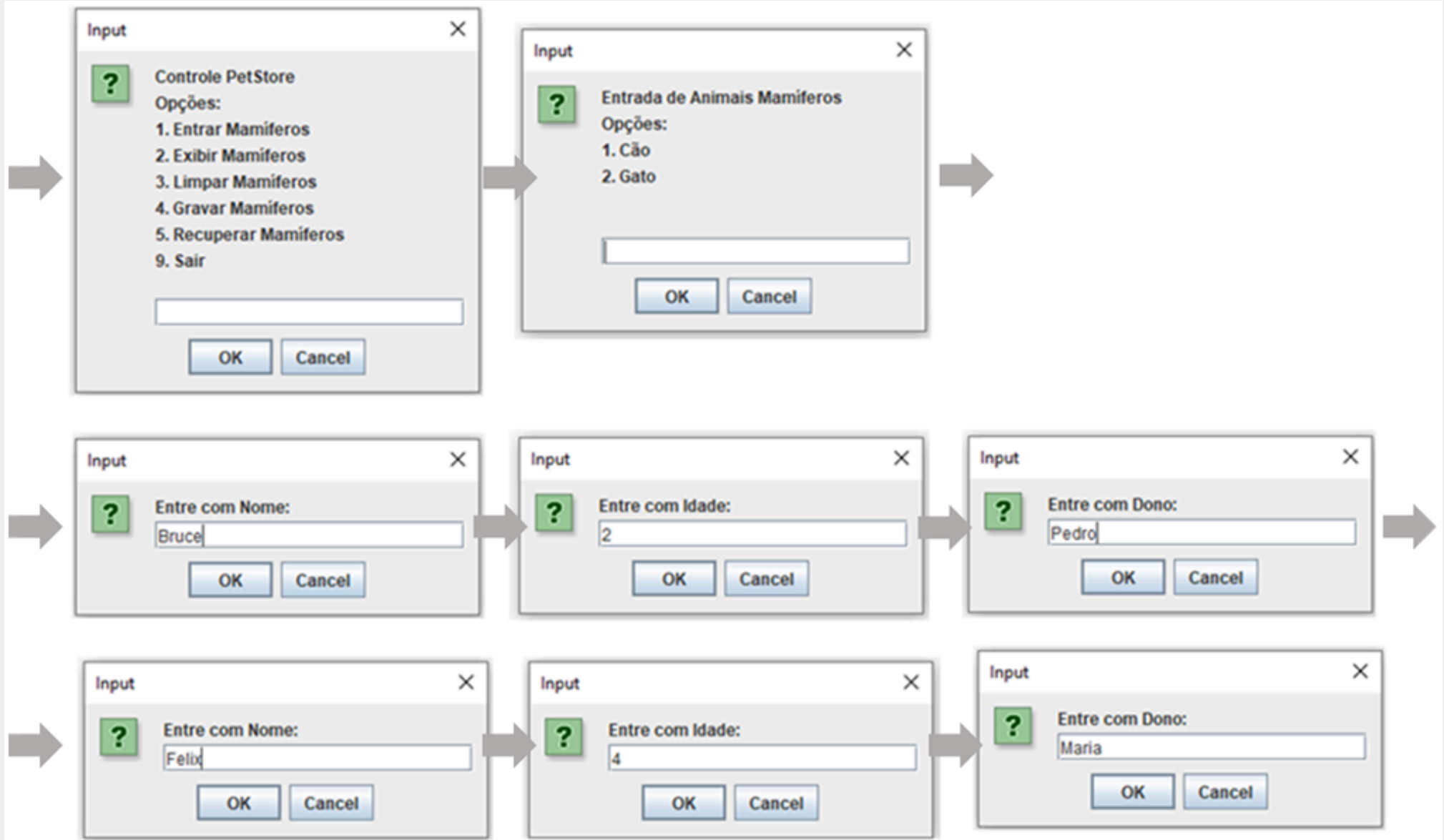
```
1 //complete o código aqui
2
3 public abstract class Mamifero /* complete o código aqui */ {
4
5     private static final long serialVersionUID = 1L;
6     //complete o código aqui
7     //complete o código aqui
8     //complete o código aqui
9     //complete o código aqui
10
11     public Mamifero(String nome, int idade, String dono) {
12         this.nome = nome;
13         this.idade = idade;
14         this.dono = dono;
15     }
16     public String toString() {
17         String retorno = "";
18         retorno += "Nome: " + this.nome + "\n";
19         retorno += "Idade: " + this.idade + " anos\n";
```

</>

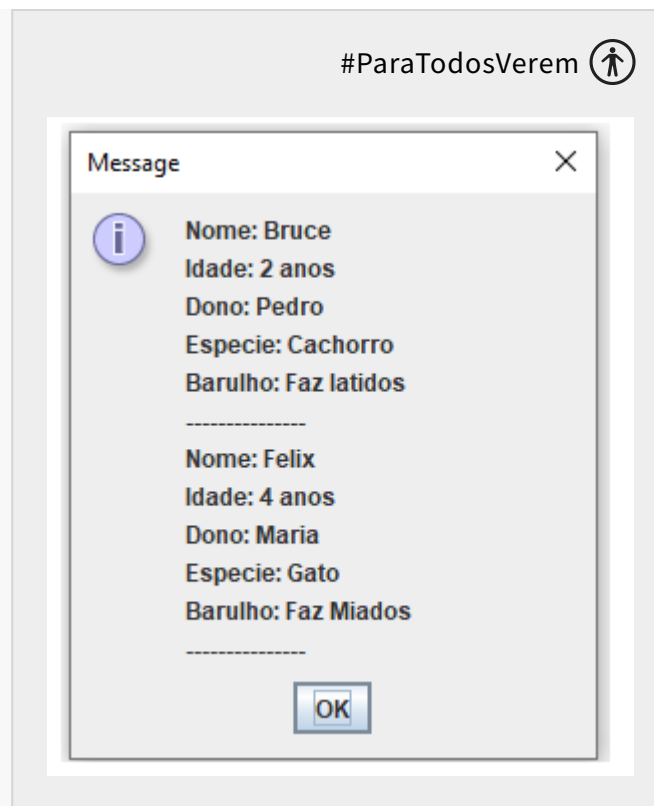
```
1 import java.io.EOFException;
2 import java.io.FileInputStream;
3 import java.io.FileNotFoundException;
4 import java.io.FileOutputStream;
5 import java.io.IOException;
6 import java.io.ObjectInputStream;
7 import java.io.ObjectOutputStream;
8 import java.util.ArrayList;
9
10 import javax.swing.JOptionPane;
11
12 public class PetStore {
13     private ArrayList<Mamifero> mamiferos;
14
15     public PetStore() {
16         this.mamiferos = new ArrayList<Mamifero>();
17     }
18     public String[] leValores (String [] dadosIn){
19         String [] dadosOut = new String [dadosIn.length];
```

Sequência de apresentação das caixas de diálogo do programa

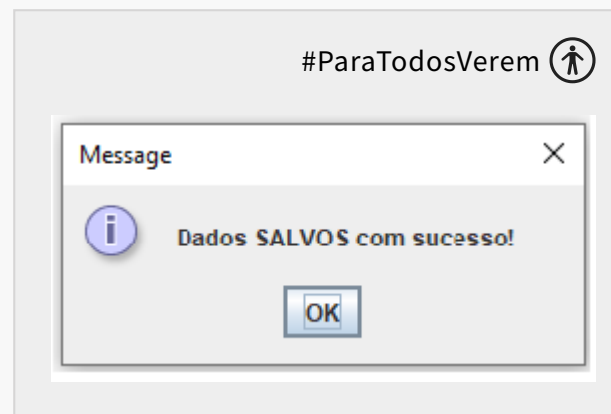
1. Entrar os dados de um cão e de um gato:



2. Exibir a lista de mamíferos:

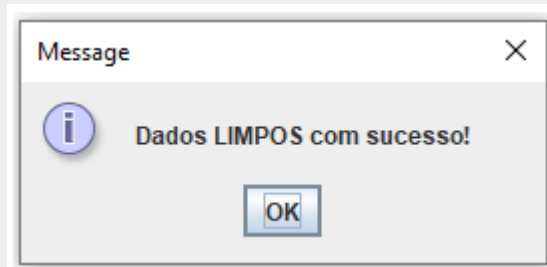


3. Gravar os objetos criados em arquivo:



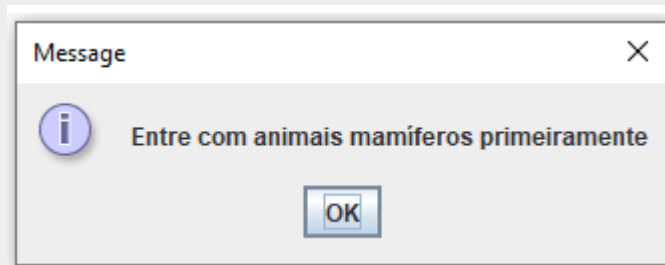
4. Limpar da memória (ArrayList) os objetos recém-criados:

#ParaTodosVerem 



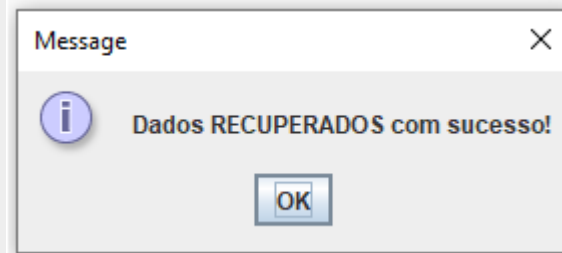
5. Exibir novamente a lista de mamíferos:

#ParaTodosVerem 



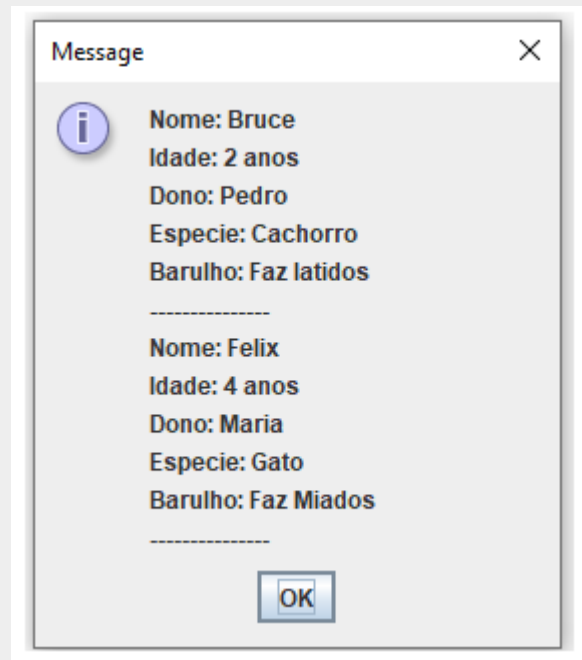
6. Recuperar os objetos persistidos:

#ParaTodosVerem 



7. Exibir novamente a lista de mamíferos:

#ParaTodosVerem 



8. Sair do programa:



Resolução do exercício



</>

```
1  import java.io.Serializable;
2
3  public abstract class Mamifero implements Serializable {
4      private String nome;
5      private int idade;
6      private String dono;
7      protected String especie;
8      ...
9      public abstract String soar();
10 }
11 -----
12 public class Cao extends Mamifero {
13
14     private static final long serialVersionUID = 1L;
15
16     public String soar() {
17         return "Faz latidos";
18     }
19     public Cao(String nome, int idade, String dono) {
```

Exercícios de fixação



EXERCÍCIO

1. Suponha que você esteja trabalhando em um sistema de gerenciamento de alunos para a PUCPR. Crie um programa que lê um arquivo contendo uma lista de alunos (nome, curso, idade) e, em seguida, permite ao usuário adicionar novos alunos à lista. Após adicionar os novos alunos, o programa deve gravar a lista atualizada em um novo arquivo.

2. Agora, imagine que você esteja desenvolvendo um sistema de gerenciamento de funcionários da Petrobras. Crie um programa que permita ao usuário inserir detalhes dos funcionários (nome, cargo, salário) por meio do teclado. Esses detalhes devem ser salvos em um objeto **funcionario**, e então, serializados em um arquivo. Depois, crie um algoritmo que leia o arquivo serializado e imprima os detalhes dos funcionários na tela.



Resolução do exercício



Exercício 1

</>

```
1  import java.io.*;
2  import java.util.*;
3
4  class Aluno {
5      String nome;
6      String curso;
7      int idade;
8
9      public Aluno(String nome, String curso, int idade) {
10         this.nome = nome;
11         this.curso = curso;
12         this.idade = idade;
13     }
14
15     @Override
16     public String toString() {
17         return nome + ", " + curso + ", " + idade;
18     }
19 }
```

Exercício 2

</>

```
1 import java.io.*;
2 import java.util.*;
3
4 class Funcionario implements Serializable {
5     private static final long serialVersionUID = 1L;
6
7     String nome;
8     String cargo;
9     double salario;
10
11     public Funcionario(String nome, String cargo, double salario) {
12         this.nome = nome;
13         this.cargo = cargo;
14         this.salario = salario;
15     }
16
17     @Override
18     public String toString() {
19         return "Nome: " + nome + ". Cargo: " + cargo + ". Salário: " + salario;
```

| Referências

GODOY, V. **Programação orientada a objetos I**. Curitiba: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – volume I**. 8. ed. São Paulo: Pearson, 2010.

SCHILD, H. **Java para iniciantes**. Porto Alegre: Bookman, 2015.

ORACLE. The Java™ Tutorials. **ORACLE**, Documentation, 2020. Disponível em:

<https://docs.oracle.com/javase/tutorial/uiswing/components/componentlist.html>. Acesso em: 20 fev. 2021.



© PUCPR - Todos os direitos reservados.